CENTRO UNIVERSITÁRIO SALESIANO DE SÃO PAULO
UNISAL – CAMPUS MARIA AUXILIADORA

Matheus Tomazella
Rafael Cesar Vanzo
Samuel dos Santos Carneiro

NOVADMIN: Módulo de Orçamento para Sistema ERP

Americana – SP
2026

Matheus Tomazella
Rafael Cesar Vanzo
Samuel dos Santos Carneiro

NOVADMIN: Módulo de Orçamento para Sistema ERP

Trabalho de Conclusão de Curso apresentado como exigência parcial para a obtenção do grau de Bacharel em Engenharia da Computação no Centro Universitário Salesiano de São Paulo.
Orientador: Prof. Dr. Leonardo Grando.

Americana – SP
2026

Matheus Tomazella
Rafael Cesar Vanzo
Samuel dos Santos Carneiro

NOVADMIN: Módulo de Orçamento para Sistema ERP

Trabalho de Conclusão de Curso apresentado como exigência parcial para a obtenção do grau de Bacharel em Engenharia da Computação no Centro Universitário Salesiano de São Paulo.
Orientador: Prof. Dr. Leonardo Grando.

Trabalho de Conclusão de Curso defendido e aprovado em ___/11/2026, pela comissão julgadora:

Prof. Dr. Leonardo Grando, UNISAL

Prof. Me. Loren, Ipsum

Americana – SP
2026

RESUMO

Apresenta a implementação de um módulo de orçamento, construído com o objetivo de centralizar dados relevantes e automatizar etapas de cálculo e a geração do documento de proposta, reduzindo o esforço necessário e aumentando a consistência do processo. O módulo foi desenvolvido com uma arquitetura em camadas, utilizando React para a interface web, Node.js com TypeScript e Express.js para a API REST, e PostgreSQL para o armazenamento de dados em um processo iterativo e incremental, com ênfase na validação contínua junto aos usuários finais e na entrega progressiva de funcionalidades. A interface organiza o processo de orçamentação em etapas sequenciais, facilitando o trabalho do usuário e garantindo flexibilidade para atender a situações reais do domínio. O resultado é um sistema que espelha fielmente o fluxo de trabalho da empresa, incorporando suas nomenclaturas, etapas e regras de negócio específicas, eliminando a necessidade de adequações operacionais forçadas.

Palavras-chave: arquitetura de software; orçamento; ERP.

ABSTRACT

Presents the implementation of a module for quotes, built to centralize relevant data and automate calculation steps and the generation of the proposal document, reducing the necessary effort and increasing the consistency of the process. The module was developed with a layered architecture, using React for the web interface, Node.js with TypeScript and Express.js for the REST API, and PostgreSQL for data storage in an iterative and incremental process, with emphasis on continuous validation with end users and progressive delivery of features. The interface organizes the quoting process into sequential steps, facilitating the user's work and ensuring flexibility to meet real-world situations in the domain. The result is a system that faithfully mirrors the company's workflow, incorporating its nomenclature, steps, and specific business rules, eliminating the need for forced operational adjustments.

Key-words: software architecture; quoting; ERP.

LISTA DE ABREVIATURAS E SIGLAS

API	Interface de Programação de Aplicações
CI/CD	Integração Contínua e Entrega Contínua
DOCX	Formato de arquivo do Microsoft Word baseado em XML
ERP	Planejamento de Recursos Empresariais (Enterprise Resource Planning)
HTML	Linguagem de Marcação de Hipertexto (HyperText Markup Language)
MER	Modelo Entidade-Relacionamento
PDF	Formato de Documento Portátil (Portable Document Format)
REST	Transferência de Estado Representacional (Representational State Transfer)
SQL	Linguagem de Consulta Estruturada (Structured Query Language)
TCC	Trabalho de Conclusão de Curso
UNISAL	Centro Universitário Salesiano de São Paulo

SUMÁRIO

1 INTRODUÇÃO 7
1.1 Contexto Empresarial 7
1.2 Objetivos 7
2 DESENVOLVIMENTO 8
2.1 Metodologia 8
2.2 Fluxo de Trabalho 9
2.3 Tecnologias e Infraestrutura 12
2.3.1 Infraestrutura e Implantação 14
2.4 Interface do Usuário 15
2.5 Resultados 22
3 CONCLUSÃO 23

REFERÊNCIAS 24

GLOSSÁRIO 25

1 INTRODUÇÃO

O processo de elaboração de orçamentos constitui atividade recorrente em empresas de engenharia e construção. Envolvendo a coleta e consolidação de informações sobre clientes, obras, materiais e regras de precificação, é um processo complexo que tem potenciais riscos significativos de inconsistência e retrabalho, podendo gerar danos à reputação e ao lucro da empresa em casos extremos.

Esse trabalho descreve a extensão de um sistema previamente construído com a implementação de um módulo especializado para geração de orçamentos e propostas de serviço. O objetivo geral é centralizar dados relevantes e automatizar etapas de cálculo e a geração do documento de proposta, reduzindo o esforço necessário e aumentando a consistência do processo.

A interface segue o molde do sistema previamente implantado, utilizando uma plataforma web clara, com edição organizada em etapas sequenciais e suporte a colaboração controlada quando aplicável.

1.1 Contexto Empresarial

O cliente é uma empresa especializada na fabricação de moldes reutilizáveis de concreto para o setor de engenharia civil. Historicamente, o processo de elaboração de propostas comerciais era executado por meio de planilhas eletrônicas e procedimentos manuais, o que gerava riscos significativos de inconsistência, dificuldade de rastreamento de decisões e reduzida capacidade de automação.

Reconhecendo essas limitações operacionais, a empresa decidiu ampliar seu sistema corporativo NOVADMIN (já em uso para outras funcionalidades) por meio da implementação de um módulo especializado para orçamentação. Esta expansão permite consolidar dados dispersos, automatizar etapas críticas e estabelecer um ponto único de controle e auditoria para o processo de proposta.

Dessa forma, diferentemente de soluções de mercado que, por mais configuráveis que sejam, impõem uma lógica genérica à qual a empresa precisa se adaptar, o módulo de orçamentação do sistema espelha fielmente o fluxo de trabalho da empresa, incorporando suas nomenclaturas, etapas e regras de negócio específicas. Essa aderência ao processo interno elimina a necessidade de adequações operacionais forçadas e garante que a ferramenta sirva à empresa, e não o contrário.

1.2 Objetivos

Os principais objetivos deste projeto são: expandir o banco de dados para suportar as entidades de fornecedor, material, orçamento, escopo, elemento, custo fixo e opção de precificação; adicionar as funcionalidades necessárias para gerenciar esses dados; automatizar o cálculo dos custos do serviço a partir das informações inseridas; automatizar a

geração do documento de proposta de serviço; e implementar um sistema de versionamento para os orçamentos.

Além disso, os objetivos não funcionais incluem oferecer uma interface simples e intuitiva, manter o estilo previamente estabelecido para o sistema, garantir flexibilidade para que o usuário não seja limitado em suas operações, priorizar a automação dos processos para reduzir a possibilidade de erro humano e aumentar a eficiência do sistema, e permitir que múltiplos usuários editem partes de um registro simultaneamente.

2 DESENVOLVIMENTO

2.1 Metodologia

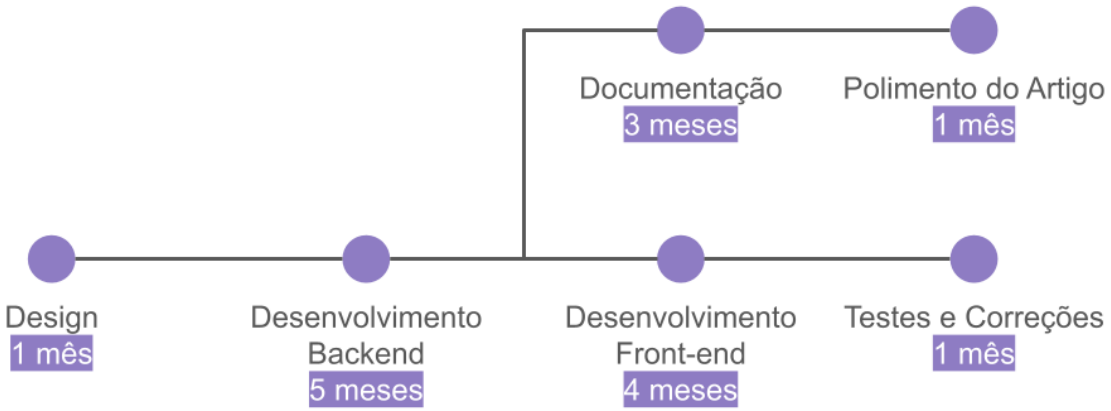
O desenvolvimento do módulo seguiu uma abordagem iterativa e incremental, com ênfase na validação contínua junto aos usuários finais e na entrega progressiva de funcionalidades.

Os requisitos funcionais foram levantados por meio de entrevistas e sessões de trabalho com a equipe operacional da empresa, garantindo que as necessidades reais do domínio fossem compreendidas e incorporadas no *design*. Durante as fases de implementação, testes unitários foram aplicados sistematicamente para validar a lógica de negócio, especialmente nas operações críticas de cálculo dos custos, utilizando dados gerados pelas planilhas substituídas pelo sistema como garantia de que os resultados continuariam condizentes com o processo.

Aspectos como a alta flexibilidade requerida pelo domínio (que demanda suporte a múltiplas exceções de regras) e a necessidade de compreender o contexto específico do cliente foram gerenciados por meio de ciclos de avaliação contínua, permitindo refinar a solução conforme o aprendizado progrediu.

O desenvolvimento foi organizado em etapas complementares. A fase inicial de *design* teve duração de um mês, seguida por cinco meses de desenvolvimento do *backend* e quatro meses de desenvolvimento do *frontend*. A documentação foi produzida em paralelo às etapas de desenvolvimento, durante três meses. No mês final, foram realizados simultaneamente os testes e correções e o polimento do artigo, conforme apresentado na Figura 1.

Figura 1 – Cronograma de desenvolvimento do projeto

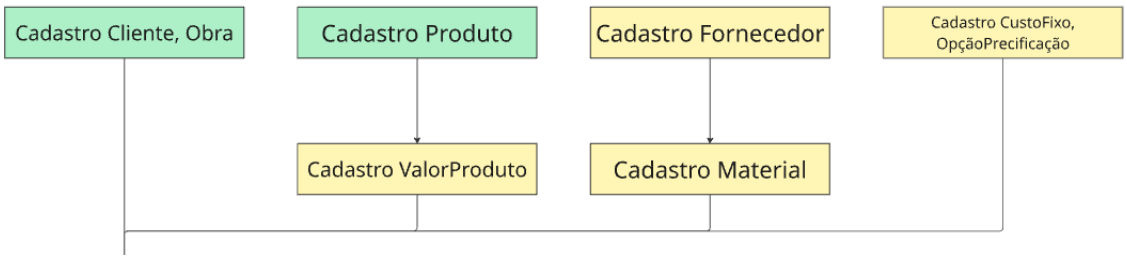


Fonte: Elaborado pelos autores, 2026.

2.2 Fluxo de Trabalho

O fluxo se inicia com a inserção de entidades que compõem o Orçamento (Figura 2): Cliente, Obra, Produto, Fornecedor, Material, Custos Fixos e Opções de Precificação. Todas são entidades simples, sem relacionamentos em sua composição, exceto por Material, que deve ser associado a um Fornecedor.

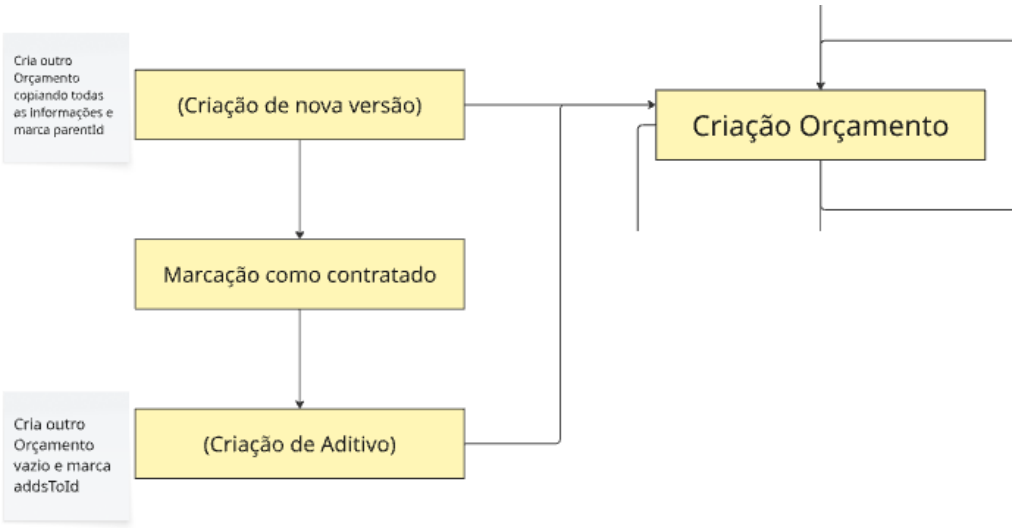
Figura 2 – Diagrama de fluxo – Criação de entidades iniciais



Fonte: Elaborado pelos autores, 2026.

Em seguida, o Orçamento é criado, sendo associado a um Cliente e a uma Obra, podendo também ser uma nova versão de um Orçamento anterior (Figura 3). Uma nova versão mantém relação com o Orçamento de origem e pode ser de dois tipos: uma revisão, caso o Orçamento ainda esteja aberto, ou um aditivo, caso já tenha sido concluído.

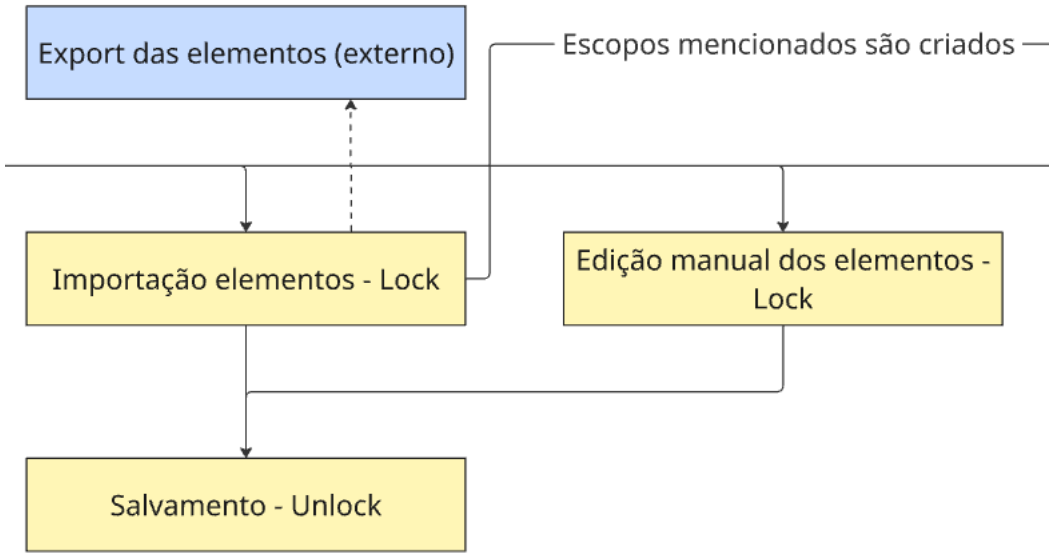
Figura 3 – Diagrama de fluxo – Criação do Orçamento



Fonte: Elaborado pelos autores, 2026.

Posteriormente, ocorre a etapa de gerenciamento dos Elementos (Figura 4), na qual o usuário pode realizar a importação por meio de arquivo, bem como efetuar edições manuais. Durante esse processo, o gerenciamento de Elementos permanece indisponível para outros usuários. A inserção de Elementos por um arquivo também permite ao usuário criar automaticamente os Escopos mencionados.

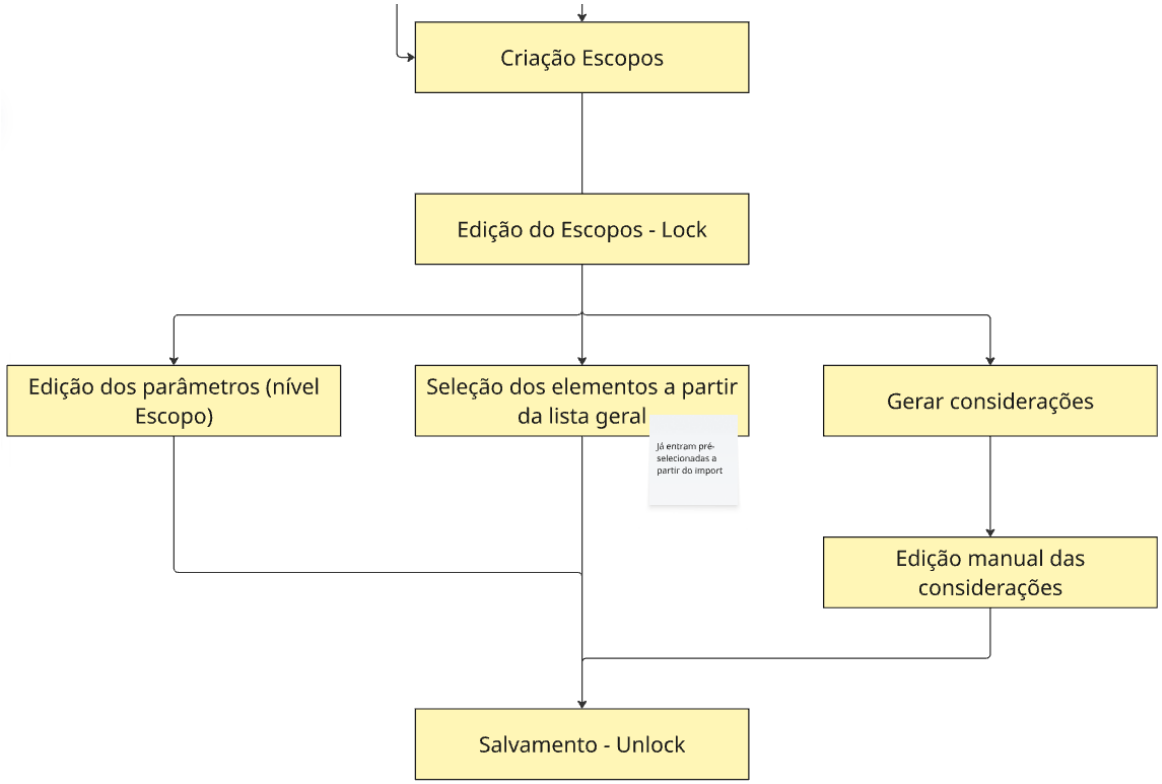
Figura 4 – Diagrama de fluxo – Gerenciamento dos Elementos



Fonte: Elaborado pelos autores, 2026.

A seguir, o gerenciamento dos Escopos (Figura 5), uma entidade que representa um trecho do projeto e contém Elementos e configurações, pode ser feito. A fim de evitar conflitos de edição, o acesso a cada Escopo é restrito a um único usuário enquanto este estiver em uso.

Figura 5 – Diagrama de fluxo – Gerenciamento dos Escopos

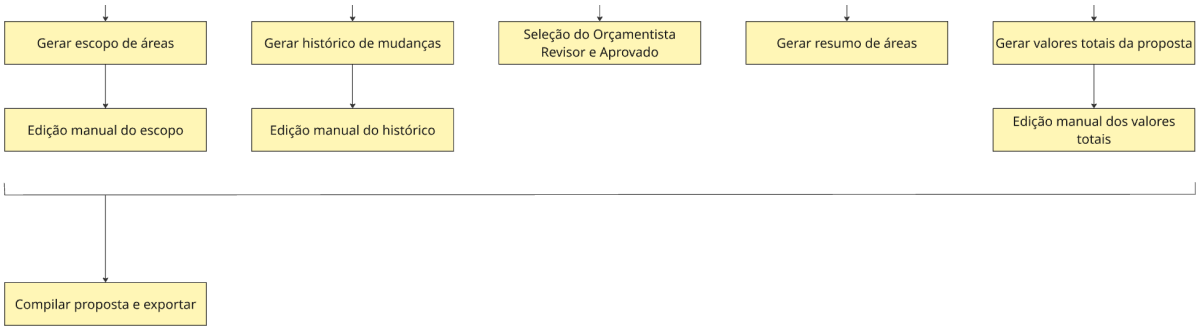


Fonte: Elaborado pelos autores, 2026.

A etapa seguinte envolve o gerenciamento de configurações, como a associação de materiais aos elementos, definição de preços de materiais e serviços e valores de metragem. Essas informações determinam o valor final da proposta e afetam como os cálculos são realizados.

Por fim, ocorre a configuração do documento (Figura 6), permitindo ao usuário ajustar textos e formatação. Após isso, os valores finais são calculados e o documento é exportado.

Figura 6 – Diagrama de fluxo – Finalização



Fonte: Elaborado pelos autores, 2026.

Um elemento importante desse processo é a inclusão de um sistema de sobrescrita de valores para determinados escopos de cálculo. Configurações como a definição dos produtos, preços de materiais e taxas de consumo de matéria-prima para determinado tipo de elemento são realizadas fora de um orçamento e servem como fonte geral, porém todas podem ser sobrescritas tanto para um determinado Orçamento quanto para um Escopo naquele Orçamento.

Isso permite que o sistema se adapte a situações reais, que não necessariamente seguem o fluxo padrão, mas ainda são de grande importância, como a aplicação de descontos ou situações incomuns como falta de funcionários ou materiais. Essa flexibilidade é um dos fatores que tornam o sistema um substituto real ao processo anterior, garantindo que se adeque à empresa e não o contrário.

2.3 Tecnologias e Infraestrutura

O módulo de orçamento foi organizado em camadas para separar a interface, as regras de negócio e a persistência dos dados. Essa divisão evita que a tela dependa diretamente da forma como o banco está estruturado e facilita a manutenção quando alguma parte do processo precisa ser ajustada.

A camada de apresentação consiste em uma interface web desenvolvida em React (Meta Platforms, Inc., s.d.), escolha realizada pela necessidade de uma navegação responsiva entre listas, formulários e abas de edição, para a qual o React oferece abstrações adequadas. Essa tecnologia também já era utilizada para o sistema já implantado, permitindo uma integração simples.

A camada de lógica de negócios consiste em uma API (Application Programming Interface) REST (Representational State Transfer) (FIELDING, 2000) implementada em Node.js (Node.js Foundation, s.d.) com TypeScript (Microsoft, s.d.) e Express.js (ExpressJS, s.d.). A API concentra as regras de domínio do módulo e expõe operações de criação, leitura, atualização, remoção e bloqueio de registros. A separação entre camadas

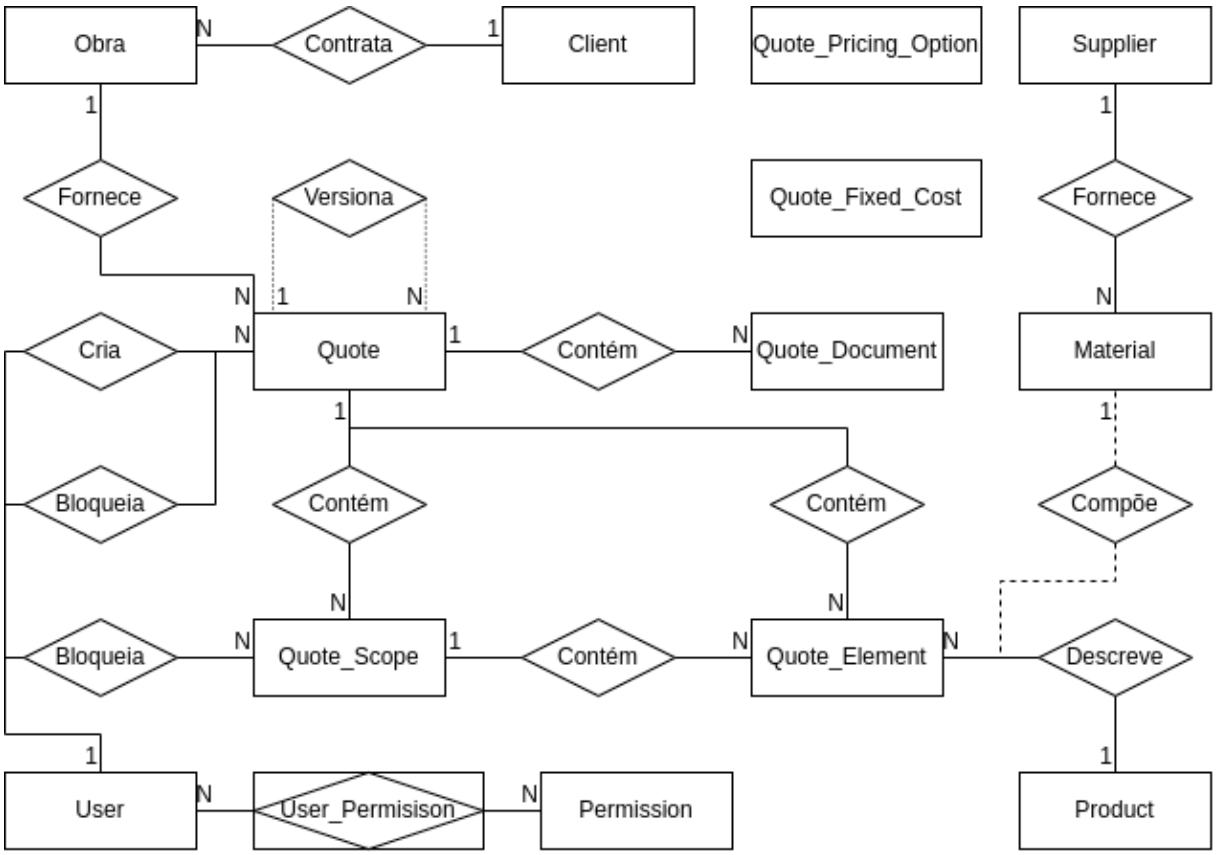
foi adotada para reduzir acoplamento, facilitar evolução independente e centralizar regras de negócio em um único ponto de controle.

Uma decisão arquitetural relevante foi a utilização de uma biblioteca compartilhada entre essas camadas, contendo regras de domínio e tipos de dados relevantes. Essa abordagem reduz duplicação de código, principalmente em cálculos, transformações de dados e tipos. Consequentemente, garante uniformidade de comportamento entre as camadas e reduz a probabilidade de divergências entre interface e processos internos.

Para armazenamento de dados foi adotado PostgreSQL (PostgreSQL Global Development Group, s.d.), um sistema gerenciador relacional que oferece suporte adequado aos requisitos de integridade referencial e transacional demandados pelo domínio. O modelo relacional favorece a preservação de consistência e facilita consultas agregadas sobre múltiplas entidades.

Para atender às necessidades do sistema, foi definido o esquema apresentado na Figura 7 por meio de um diagrama MER (Modelo Entidade-Relacionamento) (CHEN, 1976).

Figura 7 – Diagrama MER do banco de dados



Fonte: Elaborado pelos autores, 2026.

A análise da estrutura começa pelas entidades que representam elementos mais simples: clientes (*Client*), produtos (*Product*), obras, fornecedores (*Supplier*), materiais

(*Material*), usuários (*User*), opções de precificação (*Quote_Pricing_Option*) e custos fixos (*Quote_Fixed_Cost*) que possuem poucas interações entre si e no geral servem apenas para guardar dados que serão úteis para outros elementos mais complexos.

Os elementos do orçamento são armazenados na tabela Quote_Element, cujos registros se relacionam com a tabela Quote_Scope, responsável por armazenar informações dos escopos do projeto, como dados de identificação e configuração. Por sua vez, o escopo está associado à tabela Quote, que representa o Orçamento.

A tabela Quote_Document também se relaciona com o Orçamento e armazena os anexos utilizados para a criação do documento de proposta, como imagens dos projetos referenciados.

Adicionalmente, a tabela Permission armazena as permissões do sistema e se relaciona com os usuários por meio da tabela intermediária User_Permission, sendo essencial para o controle de acesso.

Essa organização permite que o sistema trate suas entidades de forma independente, facilitando a implementação de sistemas como o bloqueio de edição necessário para a edição concorrente de itens em um mesmo Orçamento.

Assim, a arquitetura foi concebida para favorecer evolução incremental. Esta abordagem reduz riscos técnicos e viabiliza entrega progressiva de funcionalidades.

2.3.1 Infraestrutura e Implantação

Para facilitar desenvolvimento, testes e implantação, o projeto adota contêineres Docker (Docker, Inc., s.d.b) e orquestração via Docker Compose (Docker, Inc., s.d.a). A utilização de contêineres garante que o ambiente de execução seja consistente entre máquinas de desenvolvimento e servidores de produção, reduzindo inconsistências. A orquestração é empregada para montar serviços dependentes (API, banco de dados, serviços complementares) com configuração unificada, simplificando a criação de ambientes reproduzíveis e isolados.

O projeto adota um banco de dados relacional (SQL) devido à natureza fortemente relacional dos dados de domínio. O PostgreSQL foi selecionado pela robustez, maturidade de funcionalidades, comunidade ativa e suporte adequado em ambientes empresariais.

Para implantações em infraestrutura local, o projeto inclui scripts e configurações de serviço que automatizam inicialização e parada do sistema com integração ao estado do sistema operacional. Esta abordagem permite reinício automático do serviço, diminuindo o tempo que fica indisponível.

Além disso, para reduzir risco em *deploys* de produção, a infraestrutura foi montada de forma a suportar um fluxo de *release Blue/Green* (FOWLER, 2010). Nesse modelo, mantém-se dois ambientes idênticos, porém enquanto um está ativo e disponível aos usuários o outro recebe a nova versão, que é testada. Após validação, o roteamento é redirecionado para o ambiente atualizado, permitindo uma recuperação rápida ao manter a

outra instância intacta caso algum problema ocorra. Com imagens Docker e orquestração, esse padrão é de simples implementação e combina bem com práticas de CI/CD para *deploys* controlados.

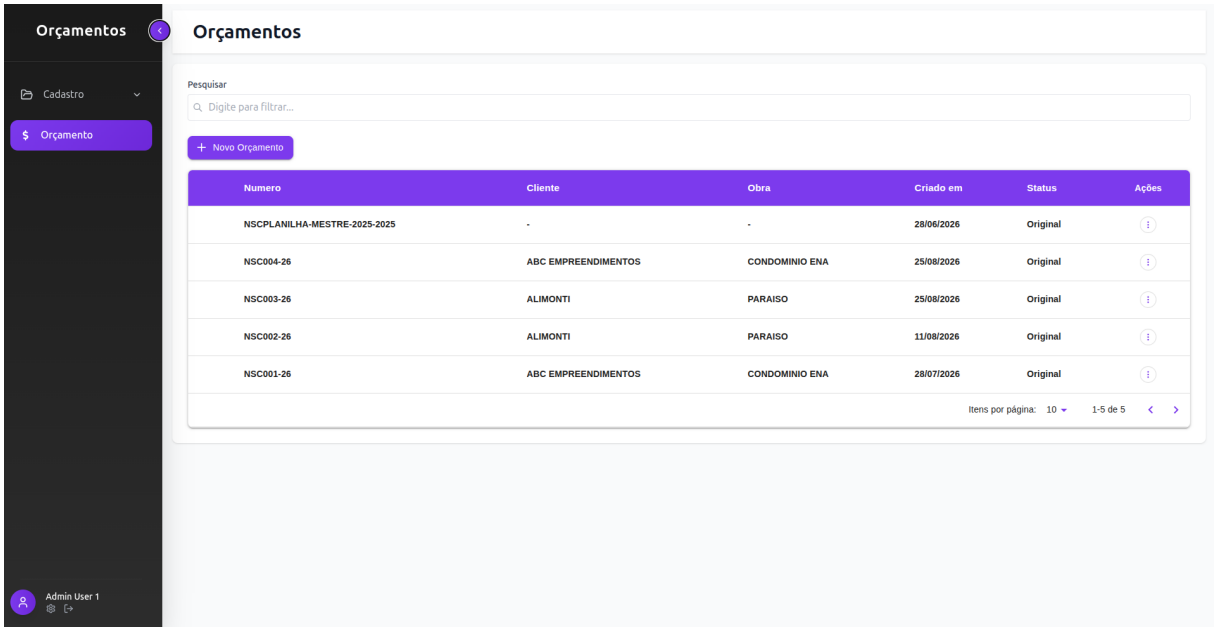
A infraestrutura também prevê centralização de logs, monitoramento básico e políticas de *backup/restore* para o PostgreSQL em múltiplos locais. A configuração por contêiner facilita a instrumentação e a padronização de métricas e alertas e o armazenamento de versões antigas do banco de dados garante recuperação em situações catastróficas com perda mínima de trabalho.

Por fim, o sistema implementa um modelo de controle de acesso baseado em permissões, associando cada usuário a papéis que determinam quais funcionalidades e dados são acessíveis. As permissões são gerenciáveis, permitindo que administradores designem papéis conforme a estrutura organizacional e responsabilidades de cada operador. Todas as operações sensíveis (criação, alteração, remoção de orçamentos e escopos) são registradas em logs de auditoria, permitindo rastreamento completo de quem fez o quê e quando.

2.4 Interface do Usuário

A interface do módulo foi concebida para organizar o processo de orçamentação em etapas sequenciais, reduzindo complexidade cognitiva e facilitando o trabalho do usuário. A tela inicial reúne as operações principais (pesquisa, filtros, criação) (Figura 8), enquanto a edição é realizada por meio de abas temáticas.

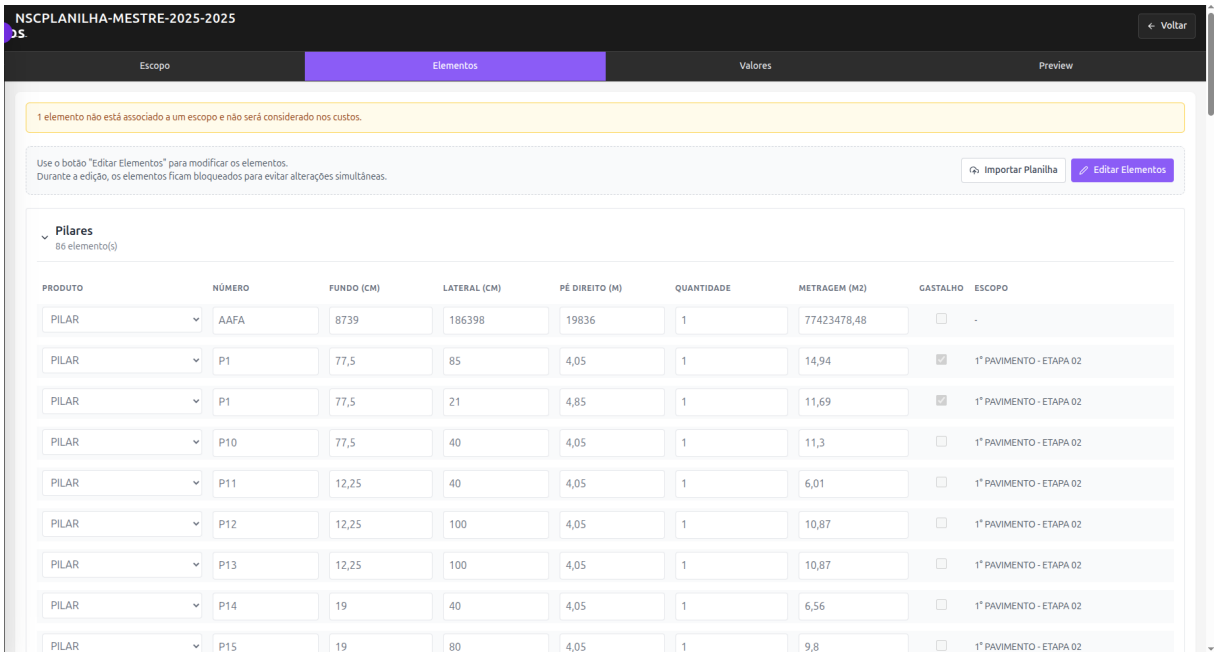
Figura 8 – Interface – Listagem de Orçamentos



Fonte: Elaborado pelos autores, 2026.

A aba Elementos permite o gerenciamento da lista de elementos do orçamento, com suporte a importação por arquivo e edição manual em lote (Figura 9). Sua interface inclui três seções para cada categoria de elemento onde os itens são mostrados em um formato de tabela, incluindo apenas os campos válidos para seu tipo.

Figura 9 – Interface – Gerenciamento de Elementos



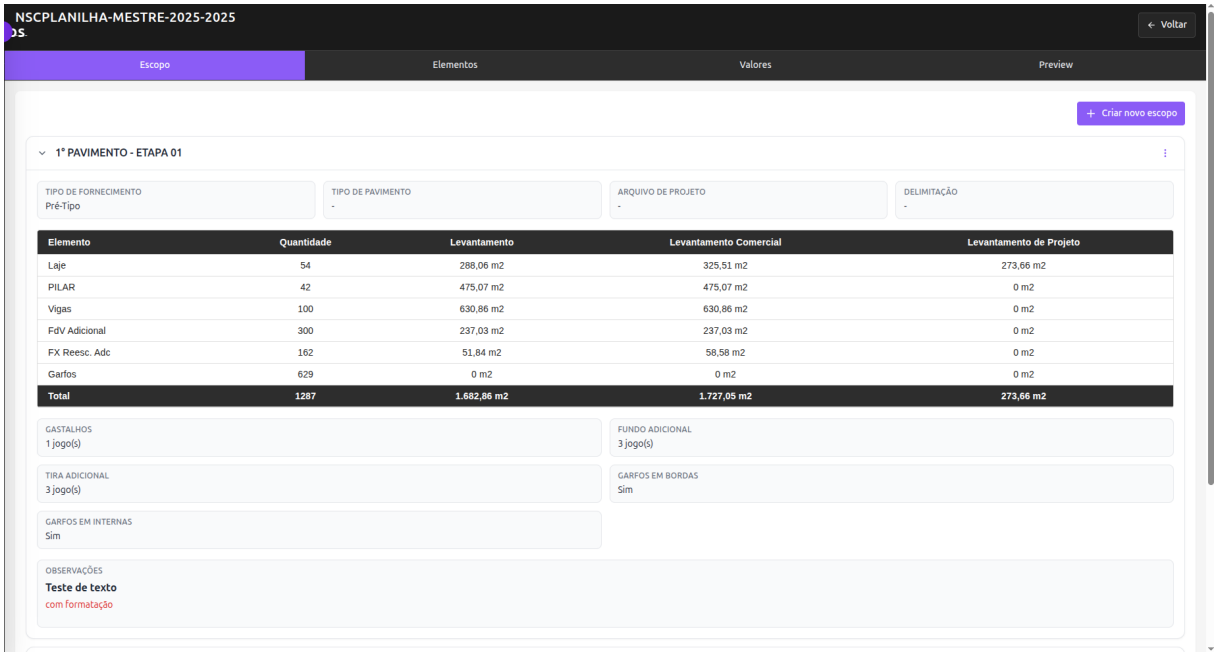
Fonte: Elaborado pelos autores, 2026.

A utilização de seções colapsáveis permite ao usuário ver apenas o que precisa, evitando a necessidade de rolar a tela por grandes distâncias, principalmente quando essa seção pode incluir centenas de itens em cada categoria. Esse elemento de *design* é muito utilizado na criação desse sistema.

O botão “Editar Elementos” inclui uma explicação sobre o bloqueio de edição da seção para outros usuários durante o uso. Essa inclusão é importante para garantir que o usuário entenda a necessidade de limitar o tempo de uso e principalmente para que não confunda a impossibilidade de editar os elementos com um erro no sistema e possua as informações necessárias para liberar seu acesso.

A aba Escopos separa os projetos em partes menores, permitindo a delimitação de grupos de elementos que podem possuir configurações diferentes (Figura 10). Cada Escopo inclui um resumo de suas informações, incluindo tanto os itens simples, como nomes de responsáveis, quanto uma tabela que resume os elementos inclusos. Isso permite ao usuário extrair uma grande quantidade de informação sem a necessidade de analisar as minúcias do Escopo.

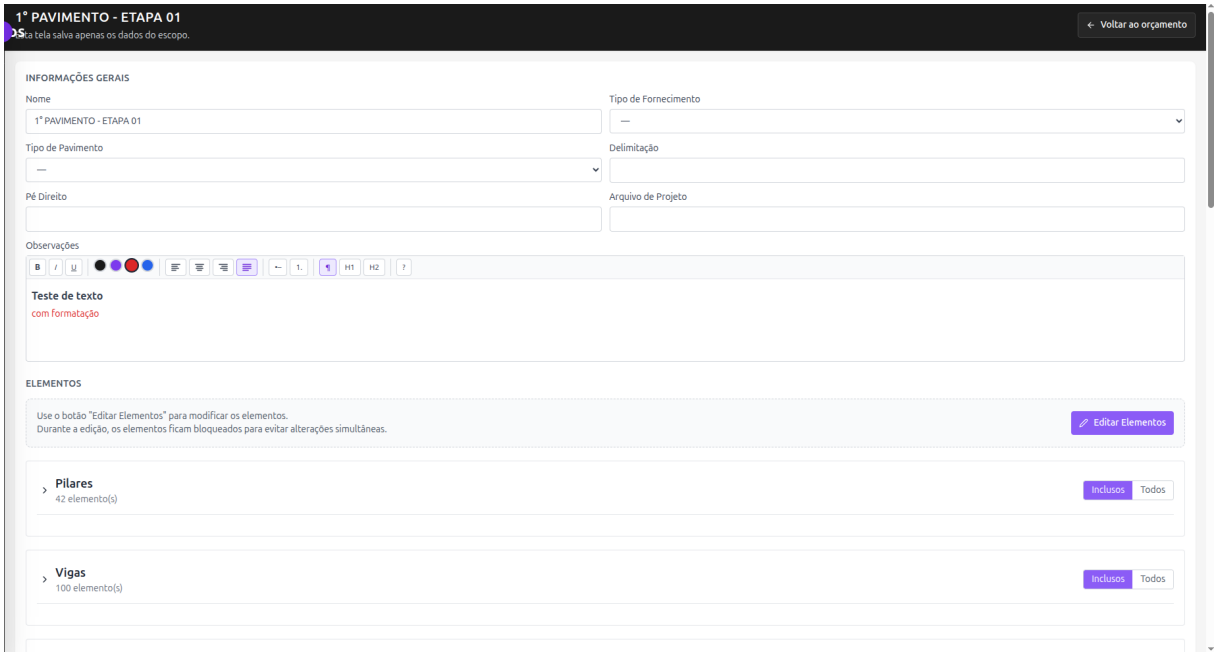
Figura 10 – Interface – Aba de listagem de Escopos



Fonte: Elaborado pelos autores, 2026.

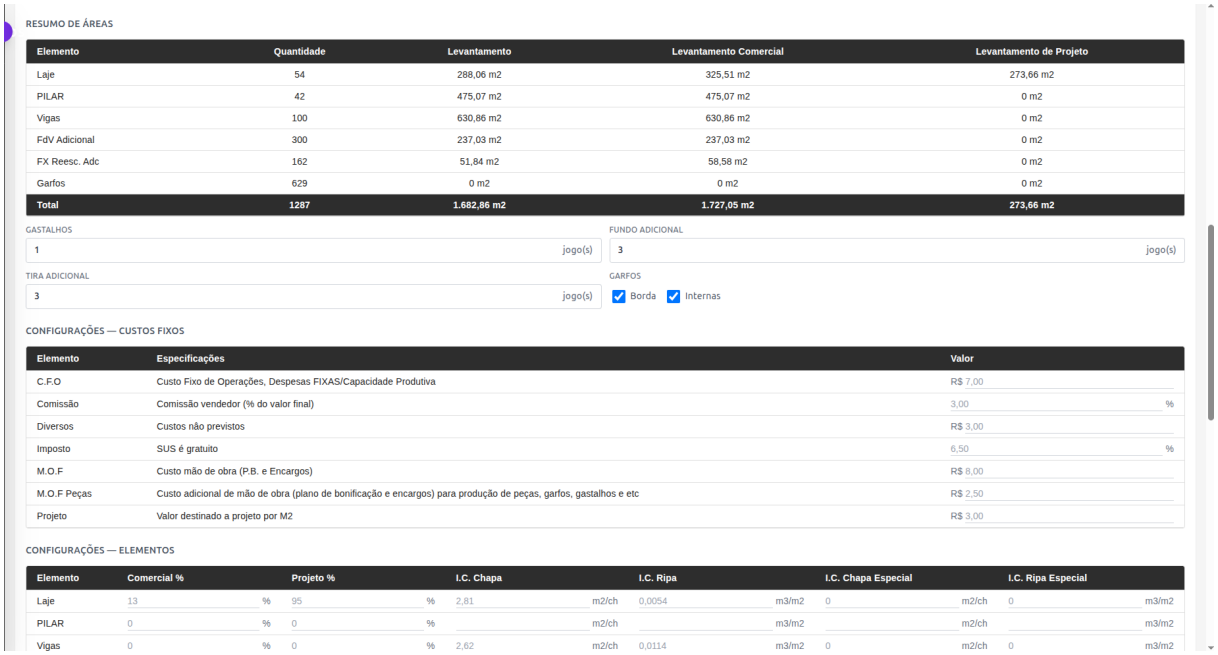
A página de edição dos Escopos agrupa o gerenciamento dos elementos e a associação de parâmetros específicos (materiais, custos fixos, taxas de uso) (Figura 11, Figura 12 e Figura 13). A separação das configurações em múltiplas seções e tabelas limita a quantidade de informação com que o usuário precisa lidar em um determinado momento, facilitando o uso da aplicação. Além disso, utilizar termos e uma estrutura similares ao que estava presente nas antigas planilhas da empresa aumenta as chances de uma transição fácil.

Figura 11 – Interface – Gerenciamento de Escopo 1



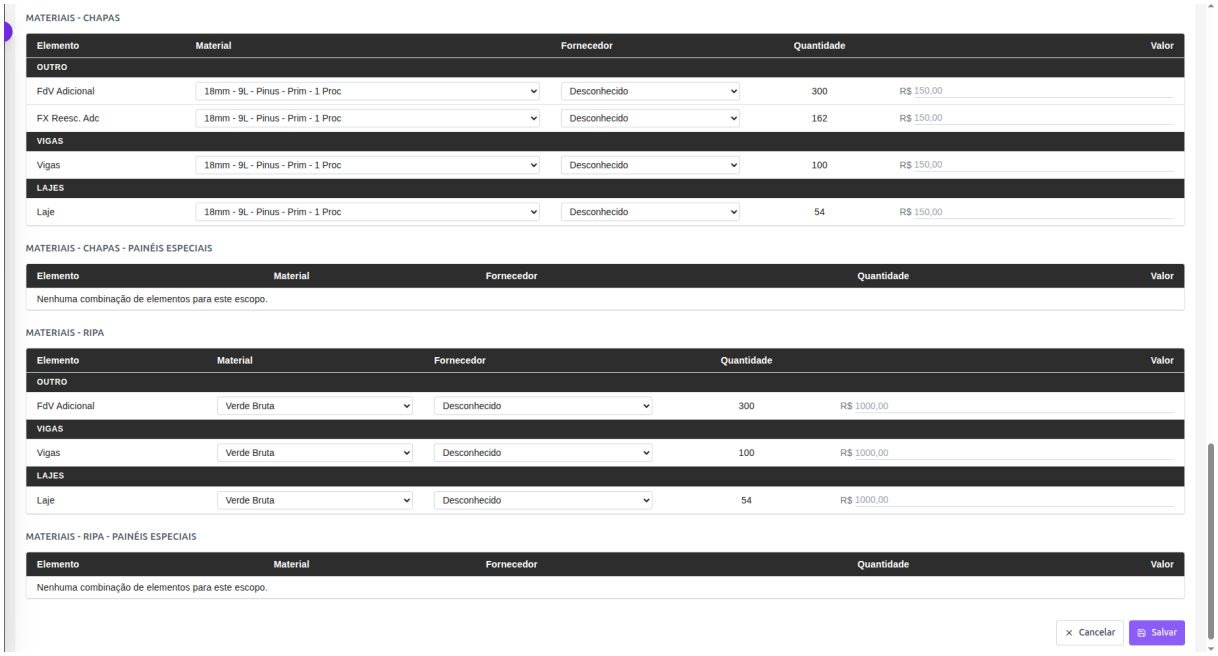
Fonte: Elaborado pelos autores, 2026.

Figura 12 – Interface – Gerenciamento de Escopo 2



Fonte: Elaborado pelos autores, 2026.

Figura 13 – Interface – Gerenciamento de Escopo 3

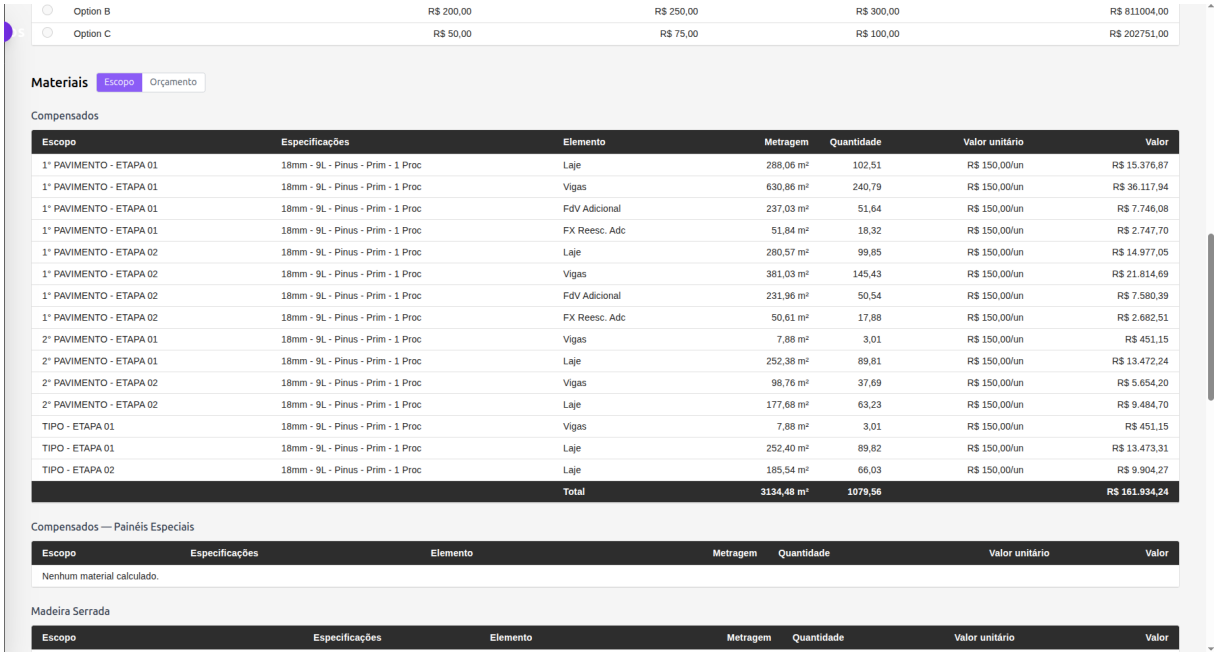


Fonte: Elaborado pelos autores, 2026.

A aba Valores consolida informações de preços, consumo de materiais e custos agregados, facilitando a visualização do impacto financeiro de cada decisão.

Todas as configurações dos escopos são agregadas, mostrando os totais de metragem e custo para os materiais, como apresentado na Figura 14. Esses valores são então somados, o resultado servindo ao usuário como uma referência para a definição do preço a ser cobrado na proposta, o que é configurado pelas opções de precificação mostradas na Figura 15.

Figura 14 – Interface – Valores – Agregação de Materiais



Option B	R$ 200,00	R$ 250,00	R$ 300,00	R$ 811004,00
Option C	R$ 50,00	R$ 75,00	R$ 100,00	R$ 202751,00

Materiais Escopo | Orçamento

Compensados

Escopo	Especificações	Elemento	Metragem	Quantidade	Valor unitário	Valor
1° PAVIMENTO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	Laje	288,06 m²	102,51	R$ 150,00/un	R$ 15.376,87
1° PAVIMENTO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	Vigas	630,86 m²	240,79	R$ 150,00/un	R$ 36.117,94
1° PAVIMENTO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	FdV Adicional	237,03 m²	51,64	R$ 150,00/un	R$ 7.746,08
1° PAVIMENTO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	FX Reesc. Adc	51,84 m²	18,32	R$ 150,00/un	R$ 2.747,70
1° PAVIMENTO - ETAPA 02	18mm - 9L - Pinus - Prim - 1 Proc	Laje	280,57 m²	99,85	R$ 150,00/un	R$ 14.977,05
1° PAVIMENTO - ETAPA 02	18mm - 9L - Pinus - Prim - 1 Proc	Vigas	381,03 m²	145,43	R$ 150,00/un	R$ 21.814,69
1° PAVIMENTO - ETAPA 02	18mm - 9L - Pinus - Prim - 1 Proc	FdV Adicional	231,96 m²	50,54	R$ 150,00/un	R$ 7.580,39
1° PAVIMENTO - ETAPA 02	18mm - 9L - Pinus - Prim - 1 Proc	FX Reesc. Adc	50,61 m²	17,88	R$ 150,00/un	R$ 2.682,51
2° PAVIMENTO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	Vigas	7,88 m²	3,01	R$ 150,00/un	R$ 451,15
2° PAVIMENTO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	Laje	252,38 m²	89,81	R$ 150,00/un	R$ 13.472,24
2° PAVIMENTO - ETAPA 02	18mm - 9L - Pinus - Prim - 1 Proc	Vigas	98,76 m²	37,69	R$ 150,00/un	R$ 5.654,20
2° PAVIMENTO - ETAPA 02	18mm - 9L - Pinus - Prim - 1 Proc	Laje	177,68 m²	63,23	R$ 150,00/un	R$ 9.484,70
TIPO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	Vigas	7,88 m²	3,01	R$ 150,00/un	R$ 451,15
TIPO - ETAPA 01	18mm - 9L - Pinus - Prim - 1 Proc	Laje	252,40 m²	89,82	R$ 150,00/un	R$ 13.473,31
TIPO - ETAPA 02	18mm - 9L - Pinus - Prim - 1 Proc	Laje	185,54 m²	66,03	R$ 150,00/un	R$ 9.904,27
		Total	**3134,48 m²**	**1079,56**		**R$ 161.934,24**

Compensados — Painéis Especiais

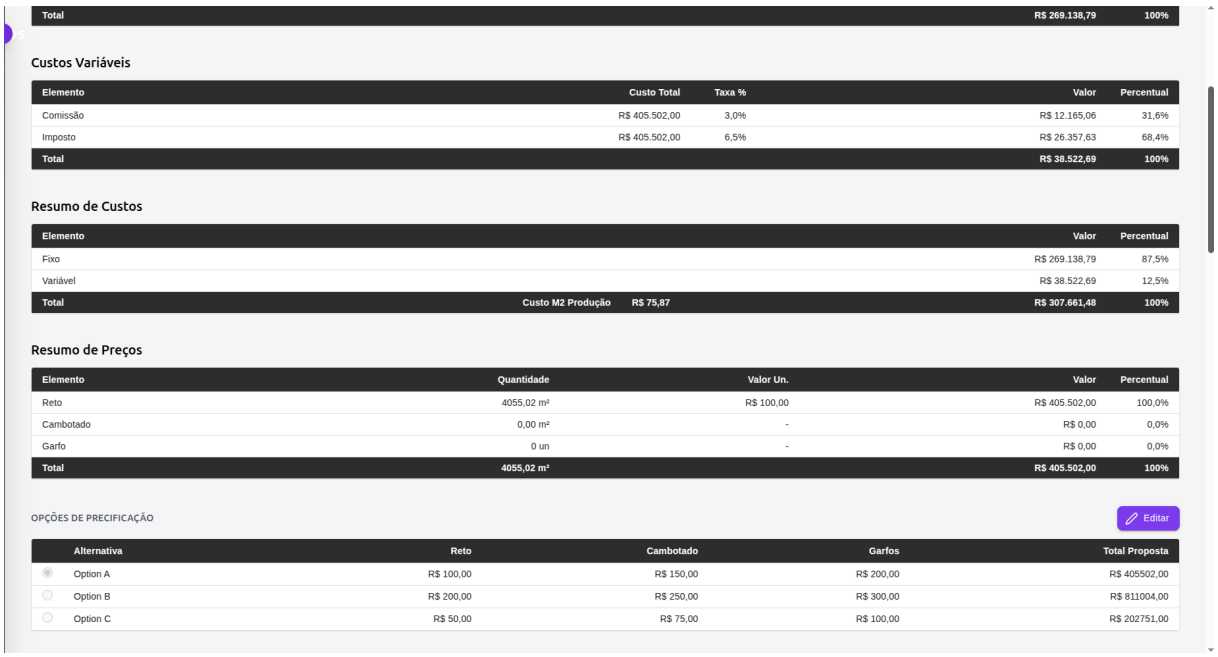
Escopo	Especificações	Elemento	Metragem	Quantidade	Valor unitário	Valor
Nenhum material calculado.						

Madeira Serrada

Escopo	Especificações	Elemento	Metragem	Quantidade	Valor unitário	Valor

Fonte: Elaborado pelos autores, 2026.

Figura 15 – Interface – Valores – Opções de Precificação



Total				**R$ 269.138,79**	**100%**

Custos Variáveis

Elemento	Custo Total	Taxa %	Valor	Percentual
Comissão	R$ 405.502,00	3,0%	R$ 12.165,06	31,6%
Imposto	R$ 405.502,00	6,5%	R$ 26.357,63	68,4%
Total			**R$ 38.522,69**	**100%**

Resumo de Custos

Elemento			Valor	Percentual
Fixo			R$ 269.138,79	87,5%
Variável			R$ 38.522,69	12,5%
Total	**Custo M2 Produção**	**R$ 75,87**	**R$ 307.661,48**	**100%**

Resumo de Preços

Elemento	Quantidade	Valor Un.	Valor	Percentual
Reto	4055,02 m²	R$ 100,00	R$ 405.502,00	100,0%
Cambotado	0,00 m²	-	R$ 0,00	0,0%
Garfo	0 un	-	R$ 0,00	0,0%
Total	**4055,02 m²**		**R$ 405.502,00**	**100%**

OPÇÕES DE PRECIFICAÇÃO — Editar

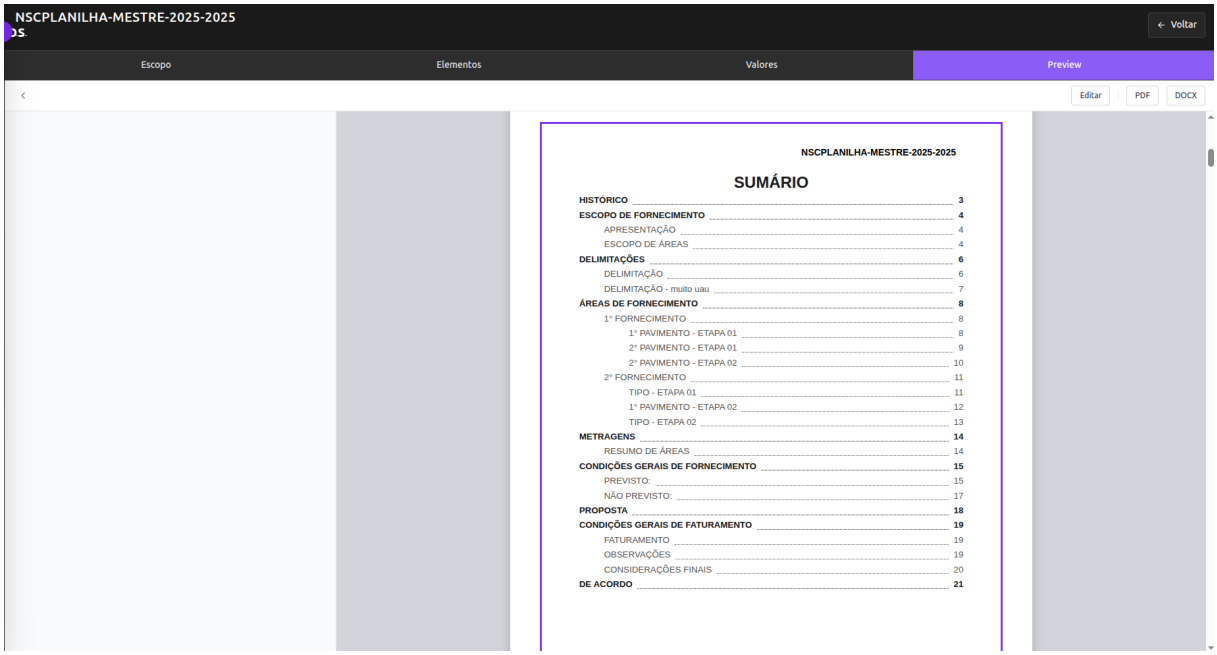
Alternativa	Reto	Cambotado	Garfos	Total Proposta
Option A	R$ 100,00	R$ 150,00	R$ 200,00	R$ 405502,00
Option B	R$ 200,00	R$ 250,00	R$ 300,00	R$ 811004,00
Option C	R$ 50,00	R$ 75,00	R$ 100,00	R$ 202751,00

Fonte: Elaborado pelos autores, 2026.

As configurações e inserções de dados são consolidados na aba *Preview*, que apresenta uma visualização prévia do documento de proposta, permitindo ajuste de texto e formatação antes da geração final. O sistema também permite adicionar imagens e inclui

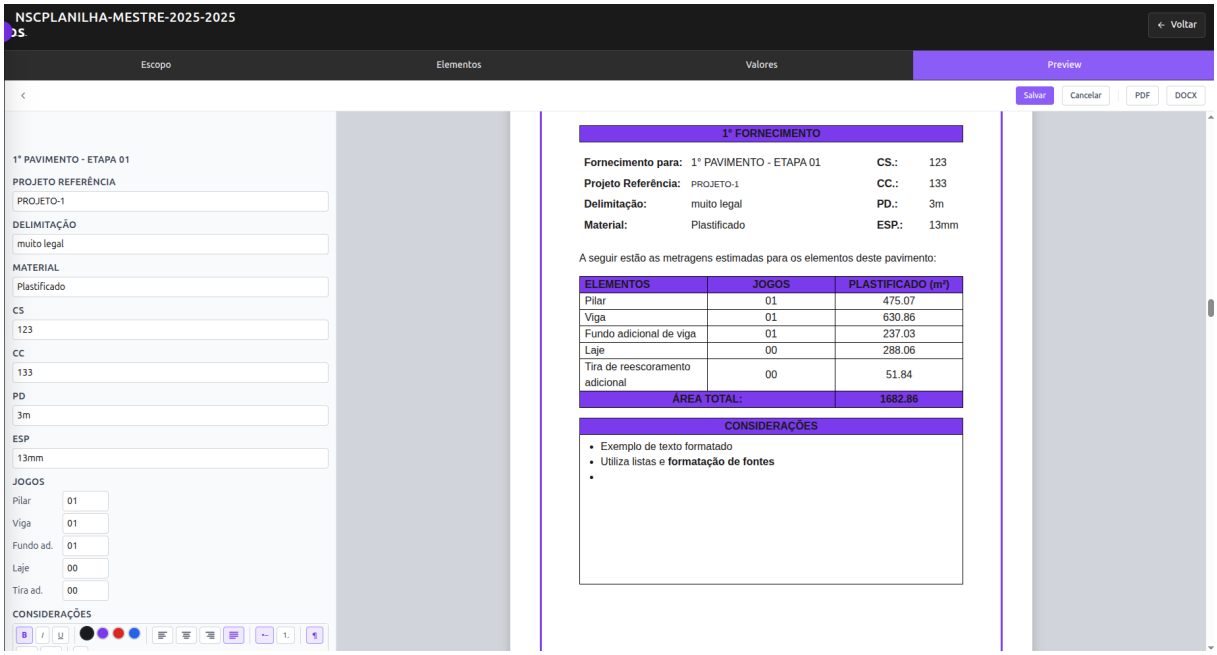
automaticamente os totais de metragem de cada escopo e os preços finais. As figuras Figura 16, Figura 17 e Figura 18 mostram alguns desses elementos.

Figura 16 – Interface – Proposta – Sumário do Documento



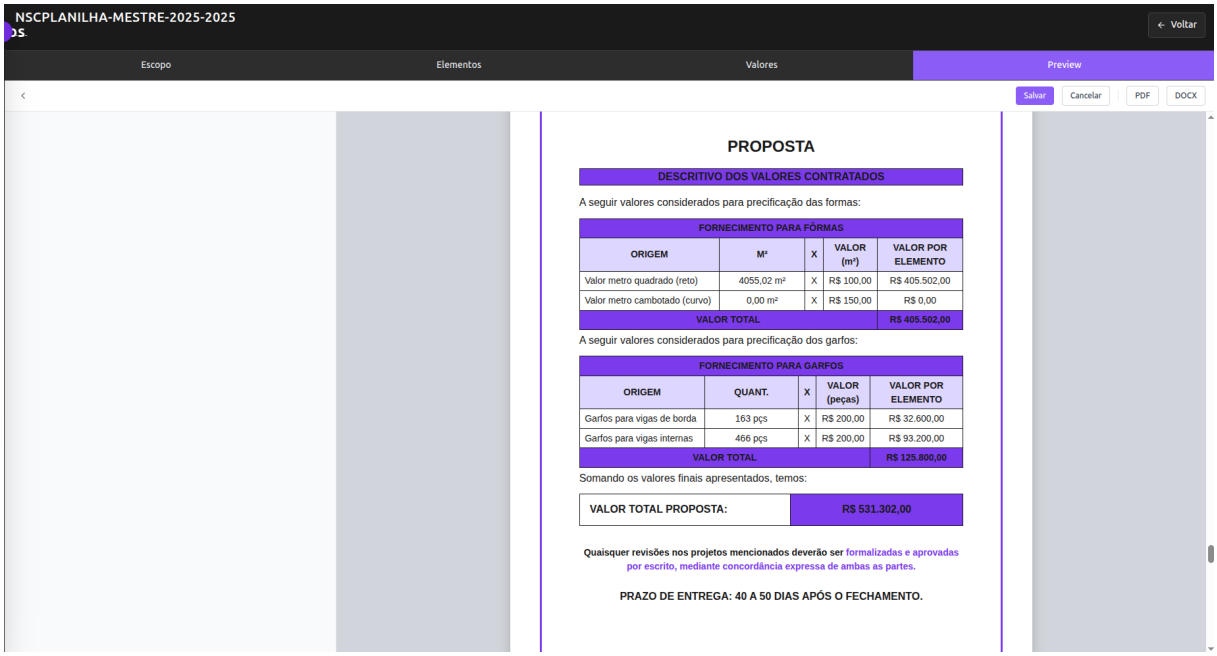
Fonte: Elaborado pelos autores, 2026.

Figura 17 – Interface – Proposta – Informações de um Escopo



Fonte: Elaborado pelos autores, 2026.

Figura 18 – Interface – Proposta – Valores da Proposta



Fonte: Elaborado pelos autores, 2026.

Após a modificação da proposta, as opções de exportação do documento por PDF e DOCX ficam disponíveis. A primeira realiza a conversão do HTML para o formato diretamente e a segunda conta com um sistema separado que reconstrói os elementos no formato de arquivo DOCX, já que uma conversão direta não é possível.

Cada etapa do fluxo foi desenhada para fornecer sinais claros ao usuário sobre o estado do documento (bloqueios, alterações pendentes) e para permitir ações atômicas e reversíveis quando necessário, buscando um produto que preza pela usabilidade, com o emprego, por exemplo, das dez heurísticas de Nielsen (NIELSEN, 1994).

2.5 Resultados

Todos os objetivos estabelecidos para o projeto foram atingidos e o trabalho realizado produziu uma base sólida e funcional para que a implementação de trabalhos futuros seja realizada.

O mecanismo de coordenação de uso por meio de bloqueio pessimista aplicado por área de edição foi implementado com sucesso, fornecendo retorno visual na interface e proteções para evitar bloqueios persistentes em caso de encerramentos inesperados. Essa funcionalidade poderá ser reutilizada em implementações futuras.

A criação de uma biblioteca contendo as regras de domínio e utilitários de negócio garante consistência entre *frontend* e *backend* e facilita testes automatizados sobre a lógica de cálculos e transformações de dados. Ela poderá ser expandida no futuro, servindo como

um local estável para manter as partes mais importantes do sistema, garantindo segurança ao desenvolvedor.

3 CONCLUSÃO

O módulo de orçamento foi concluído, atingindo os objetivos estabelecidos: centralização de dados, organização do fluxo em unidades de trabalho controladas com mecanismos de coordenação, mecanismos de gerenciamento de dados, cálculo de totais e geração do documento final. A arquitetura adotada demonstrou-se adequada para proporcionar coerência lógica e agilidade de desenvolvimento.

Os benefícios práticos observados incluem consolidação de processos anteriormente dispersos, maior segurança em edições colaborativas por meio de bloqueios coordenados, e preservação de histórico de revisões, além da automação dos processos. Estes fatores contribuem diretamente para redução de retrabalho e melhoria da rastreabilidade.

Durante o desenvolvimento, um dos desafios principais identificados foi a compreensão do contexto do cliente, já que o domínio de orçamentação na empresa apresenta complexidades que vão além de processos padrão de engenharia. Compreender plenamente o contexto, as exceções às regras e as necessidades específicas da operação demandou sessões iterativas com o cliente, enfatizando a importância de validação contínua.

Para atender às necessidades desse processo, o sistema foi projetado para ser altamente flexível, permitindo múltiplas exceções. Isso gerou desafios técnicos na modelagem de dados e lógica de validação. A solução adotada, baseada na definição de mecanismos de sobrescrita de valores para determinados escopos de uso, demonstrou ser eficaz, mas gera grande complexidade e compromete a manutenibilidade do projeto.

A resolução de problemas, tanto de implementação quanto decorrentes de mudanças solicitadas pelos usuários, é natural e deve ser conduzida de forma contínua. Embora o projeto esteja completo, trabalhos futuros poderão incluir a evolução das funcionalidades e a manutenção do sistema.

REFERÊNCIAS

CHEN, P. P.-S. The entity-relationship model: toward a unified view of data. *ACM Transactions on Database Systems*, New York, v. 1, n. 1, p. 9–36, mar. 1976. Disponível em: <https://dl.acm.org/doi/10.1145/320434.320440>. Acesso em: 20 maio 2026. Citado na página 13.

Docker, Inc. *Docker Compose: documentação oficial*. [S.l.]: Docker, Inc., s.d. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 20 maio 2026. Citado na página 14.

Docker, Inc. *Docker: documentação oficial*. [S.l.]: Docker, Inc., s.d. Disponível em: <https://docs.docker.com/>. Acesso em: 20 maio 2026. Citado na página 14.

ExpressJS. *Express: documentação oficial*. OpenJS Foundation, s.d. Disponível em: <https://expressjs.com/>. Acesso em: 20 maio 2026. Citado na página 12.

FIELDING, R. T. *Architectural styles and the design of network-based software architectures*. Tese (Tese (Doutorado em Informática)) — University of California, Irvine, Irvine, 2000. Disponível em: <https://www.ics.uci.edu/ fielding/pubs/dissertation/top.htm>. Acesso em: 20 maio 2026. Citado na página 12.

FOWLER, M. *BlueGreenDeployment*. martinfowler.com, 2010. Disponível em: <https://martinfowler.com/bliki/BlueGreenDeployment.html>. Acesso em: 20 maio 2026. Citado na página 14.

Meta Platforms, Inc. *React: documentação oficial*. Meta Platforms, Inc., s.d. Disponível em: <https://react.dev/>. Acesso em: 20 maio 2026. Citado na página 12.

Microsoft. *TypeScript: documentação oficial*. Microsoft Corporation, s.d. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 20 maio 2026. Citado na página 12.

NIELSEN, J. *10 usability heuristics for user interface design*. Fremont: Nielsen Norman Group, 1994. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Acesso em: 20 maio 2026. Citado na página 22.

Node.js Foundation. *Node.js: documentação oficial*. OpenJS Foundation, s.d. Disponível em: <https://nodejs.org/en/docs/>. Acesso em: 20 maio 2026. Citado na página 12.

PostgreSQL Global Development Group. *PostgreSQL: documentação oficial*. s.d. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 20 maio 2026. Citado na página 13.

GLOSSÁRIO

API – Application Programming Interface. Interface que permite a comunicação entre dois sistemas de software.

Backend – Camada de lógica e dados de um sistema, responsável pelo processamento e armazenamento das informações, sem interação direta com o usuário.

Bloqueio Pessimista – Estratégia de controle de concorrência que reserva exclusividade sobre um recurso antes de modificá-lo, prevenindo alterações simultâneas por outros usuários.

CI/CD – Continuous Integration/Continuous Delivery (Integração e Entrega Contínuas). Conjunto de práticas que automatizam a integração, teste e entrega de software ao longo do desenvolvimento.

Docker – Plataforma de conteinerização que permite empacotar aplicações e suas dependências em ambientes isolados e reproduzíveis chamados contêineres.

Docker Compose – Ferramenta complementar ao Docker que permite definir e orquestrar múltiplos contêineres com uma única configuração, simplificando a criação de ambientes complexos.

Express.js – Framework web para Node.js utilizado para a construção de APIs e servidores HTTP.

Frontend – Camada de apresentação de um sistema, responsável pela interface visual e pela interação direta com o usuário.

Modal – Elemento de interface que exibe conteúdo em sobreposição à tela principal, bloqueando a interação com o restante da aplicação até ser fechado ou confirmado.

Node.js – Ambiente de execução JavaScript no lado do servidor, utilizado para construção de APIs e aplicações de rede.

REST – Representational State Transfer. Estilo arquitetural para APIs que define restrições sobre como os serviços devem se comunicar via HTTP.

Transação – Operação indivisível executada completamente ou revertida em sua totalidade

(tudo ou nada), garantindo a integridade dos dados.

TypeScript – Linguagem de programação baseada em JavaScript que adiciona tipagem ao código, aumentando a segurança e a previsibilidade do desenvolvimento.

Versionamento – Processo de manter e rastrear múltiplas versões de um registro, preservando as relações entre elas para fins de auditoria e rastreabilidade.